

แบบทดสอบท้ายแล็บ

Custom ViewResolver ใน Spring Boot + Thymeleaf

วิชา CP353002 Principles of Software Design

ชื่อ-นามสกุล: ปิยพันธ์ แก้วเก็บคำ รหัสนักศึกษา: 673380413-9

คำชี้แจง: ส่วนที่ 1 ปรนัย เลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว ส่วนที่ 2 อัตนัย ตอบเป็นข้อความอธิบาย ส่วนที่ 3 ปฏิบัติ
ให้ลงมือแก้ไขได้จริงในโปรแกรมของตัวเอง แล้ว capture หน้าจอผลลัพธ์ปะในกล่องที่เตรียมไว้

ส่วนที่ 1 — ปรนัย (Multiple Choice)

1. ในสถาปัตยกรรม MVC ส่วนใดที่ทำหน้าที่แปลง "ชื่อ view" ซึ่งตรงกระให้กลายเป็น path ของไฟล์ view จริง?

(2 คะแนน)

- ก. Model
- ข. Controller
- ☒ ค. ViewResolver
- ง. DispatcherServlet

2. เมื่อ Controller เขียน return "home"; ค่า "home" ที่ถูกส่งกลับคืออะไร? (2 คะแนน)

- ก. Path ของไฟล์ HTML บนเครื่อง server
- ☒ ข. ชื่อ view ซึ่งตรงกระ (logical view name)
- ค. ชื่อ bean ของ ViewResolver
- ง. ชื่อ method ของ Controller

3. component ใดใน Spring MVC เป็นผู้เรียกใช้ ViewResolver โดยอัตโนมัติ หลังจาก Controller คืนค่าชื่อ view? (2 คะแนน)

- ก. HandlerMapping
- ☒ ข. DispatcherServlet
- ค. SpringTemplateEngine
- ง. HomeController

4. ใน ThymeleafConfig.java เมธอด resolver.setPrefix("classpath:/custom-templates/") ทำหน้าที่อะไร? (2

คะแนน)

- ก. กำหนด URL ที่ Controller จะรับ request
- ☒ ข. กำหนดโฟลเดอร์ต้นทางที่ ViewResolver จะค้นหาไฟล์ template

ค. กำหนดชื่อ database connection

ง. กำหนด port ที่แอปพลิเคชันรัน

5. ถ้ามีการลงทะเบียน ViewResolver มากกว่าหนึ่งตัวในบริบท (context) เดียวกัน Spring จะเลือกใช้ตัวไหนก่อน? (2 คะแนน)

ก. ตัวที่ประกาศเป็นครั้งแรกในไฟล์เสมอ

☒ ข. ตัวที่มีค่า order ต่ำกว่า (ถูกเรียกก่อน)

ค. สุ่มเลือก

ง. ตัวที่ตั้งชื่อ bean เรียงตามตัวอักษร

6. เพราะเหตุใด Controller ในตัวอย่างนี้จึงใช้ @Controller แทน @RestController? (2 คะแนน)

ก. เพราะ @RestController ใช้กับ Thymeleaf ไม่ได้เลย

☒ ข. เพราะต้องการให้ค่าที่ return ถูกตีความเป็นชื่อ view ไม่ใช่ response body โดยตรง

ค. เพราะ @Controller ทำงานเร็วกว่า

ง. ไม่มีความแตกต่างกันเลย

ส่วนที่ 2 — อัตนัย (Short Answer / Essay)

1. ใครเป็นผู้เรียกใช้ Custom ViewResolver และเรียกเมื่อไร? จงอธิบายเป็นลำดับขั้นตอน ตั้งแต่ Browser ส่ง request จนได้ HTML response กลับ (5 คะแนน)

1.1 **Browser ส่ง Request:** ผู้ใช้งานส่ง HTTP Request (เช่น GET /) มายัง Web Server

1.2 **DispatcherServlet รับงาน:** DispatcherServlet รับ Request เข้ามา แล้วสอบถาม HandlerMapping เพื่อหาว่า Request นี้ต้องส่งไปให้ Controller ตัวไหนประมวลผล

1.3 เรียก **Controller:** DispatcherServlet ส่งการทำงานไปยัง Controller (เช่น HomeController)

1.4 **Controller ประมวลผลและส่งคืนค่า:** Controller ทำงาน จัดเตรียมข้อมูลใส่ Model แล้วส่งคืนค่าชื่อ view เจ็ตรรกะ (เช่น return "home") กลับมายัง DispatcherServlet

1.5 เรียกใช้ **Custom ViewResolver:** DispatcherServlet นำชื่อ "home" ส่งต่อให้ **Custom ViewResolver** เพื่อแปลงเป็น Path ของไฟล์จริง

1.6 ค้นหาไฟล์ **Template:** Custom ViewResolver นำ Prefix (classpath:/custom-templates/) และ Suffix (.html) มาประกอบกับชื่อ view จนได้ Path จริงคือ classpath:/custom-templates/home.html

1.7 **Render HTML:** Thymeleaf Template Engine นำข้อมูลจาก Model ไปประมวลผลร่วมกับไฟล์ HTML template เพื่อสร้างเป็นหน้า HTML สมบูรณ์

1.8 **ส่ง HTML Response:** DispatcherServlet นำโค้ด HTML ที่ประมวลผลเสร็จแล้ว ส่งกลับไปให้ Browser ของผู้ใช้แสดงผลบนหน้าจอ

2. จงอธิบายว่าเหตุใดการแยก ViewResolver ออกจาก Controller จึงสอดคล้องกับหลักการ separation of concerns และ dependency inversion ใน Principles of Software Design (5 คะแนน)

1. Separation of Concerns (SoC) — การแยกความรับผิดชอบ

หลักการนี้ระบุว่า ซอฟต์แวร์ควรแบ่งออกเป็นส่วนๆ โดยแต่ละส่วนทำหน้าที่รับผิดชอบเฉพาะเรื่องของตัวเอง

- หน้าที่ของ **Controller:** สนใจเฉพาะ **Business & Flow Control Logic** เช่น รับ Request, ประมวลผลข้อมูล, จัดเตรียม Model และตัดสินใจว่า "จะแสดงผลด้วย View ชื่ออะไร" (คืนค่าเป็นแค่ Logical View Name เช่น "home")
- หน้าที่ของ **ViewResolver:** สนใจเฉพาะ **Presentation & Technical Details** เช่น "จะไปหาไฟล์นั้นจากที่ไหน" (การต่อ Prefix/Suffix, การค้นหา Path จริงของไฟล์ .html และการส่งประมวลผล Template Engine)

ประโยชน์: หากอนาคตต้องการย้ายไฟล์เตอร์เก็บไฟล์ HTML จาก custom-templates/ ไปที่อื่น หรือเปลี่ยนไปใช้ Template Engine ตัวอื่น (เช่น FreeMarker) เราสามารถแก้ไขได้ที่ตัวตั้งค่า ViewResolver เพียงจุดเดียว โดยไม่ต้องแก้ไขโค้ดใน Controller เลยแม้แต่บรรทัดเดียว

2. Dependency Inversion Principle (DIP) — การกลับทิศทางการพึ่งพา

หลักการนี้ระบุว่า โมดูลระดับสูง (High-level Module) ไม่ควรขึ้นอยู่กับโมดูลระดับต่ำ (Low-level Module) แต่ทั้งคู่ควรขึ้นอยู่กับ **Abstraction** (ข้อตกลงส่วนกลาง)

- ความสัมพันธ์:
 - Controller (High-level Module) ไม่ได้สร้างหรือขึ้นตรงกับ Object ของไฟล์ HTML หรือตัวแปลงไฟล์อย่าง ThymeleafViewResolver (Low-level Details) โดยตรง
 - แต่ Controller สื่อสารผ่าน **Abstraction** นั่นคือการส่งคืนข้อความธรรมดา (String เช่น "home") ซึ่งเป็นตัวแทนเชิงตรรกะ
 - การทำงาน: ทั้ง Controller และ ViewResolver ทำงานร่วมกันผ่าน Framework สัญญาากลางของ Spring MVC (ผ่าน Interface ViewResolver) ทำให้ Controller หลุดจากการผูกติด (Decoupled) กับเทคโนโลยีและโครงสร้างไฟล์ฝั่งหน้าบ้านอย่างสิ้นเชิง
-

3. หากต้องการเพิ่ม view ใหม่ชื่อ "about" โดยยังใช้ custom ViewResolver ตัวเดิม
ต้องสร้างหรือแก้ไขไฟล์ใดบ้าง? ระบุชื่อไฟล์และ path ให้ครบถ้วน (3 คะแนน)

1. แก้ไขไฟล์ Controller (เพิ่ม Route)

- Path ของไฟล์: src/main/java/com/example/demo/controller/HomeController.java
 - สิ่งที่ต้องทำ: เพิ่ม method ใหม่ภายในคลาสเพื่อรับ Request (เช่น /about) และส่งคืนค่าชื่อ View เชิงตรรกะเป็น "about"
-

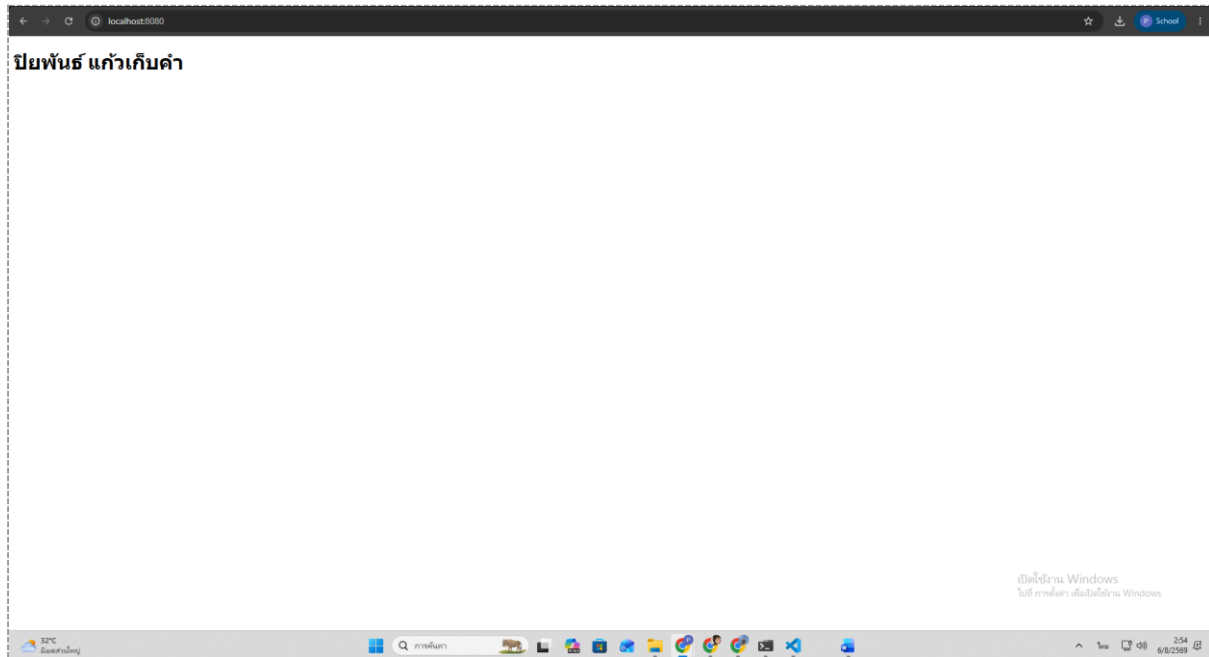
2. สร้างไฟล์ Template HTML ใหม่

- Path ของไฟล์: src/main/resources/custom-templates/about.html
 - สิ่งที่ต้องทำ: สร้างไฟล์ about.html ไว้ในโฟลเดอร์ custom-templates (ซึ่งเป็น Prefix ที่ตั้งค่าไว้ใน Custom ViewResolver)
-

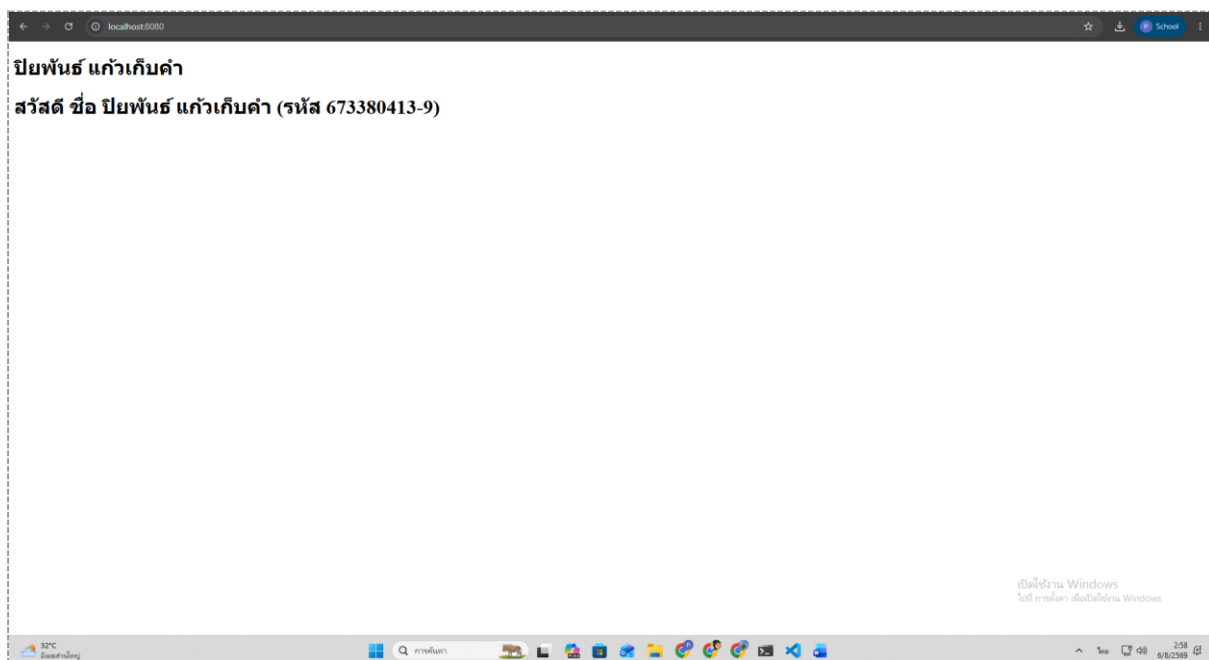
ส่วนที่ 3 — ปฏิบัติ (แก้โค้ดจริง + แบนภาพหน้าจอ)

ทำตามคำสั่งแต่ละข้อในโปรเจกต์ของตัวเอง รันด้วย `mvn spring-boot:run` แล้ว capture หน้าจอผลลัพธ์แปะในกล่องเส้นประด้านล่างแต่ละข้อ

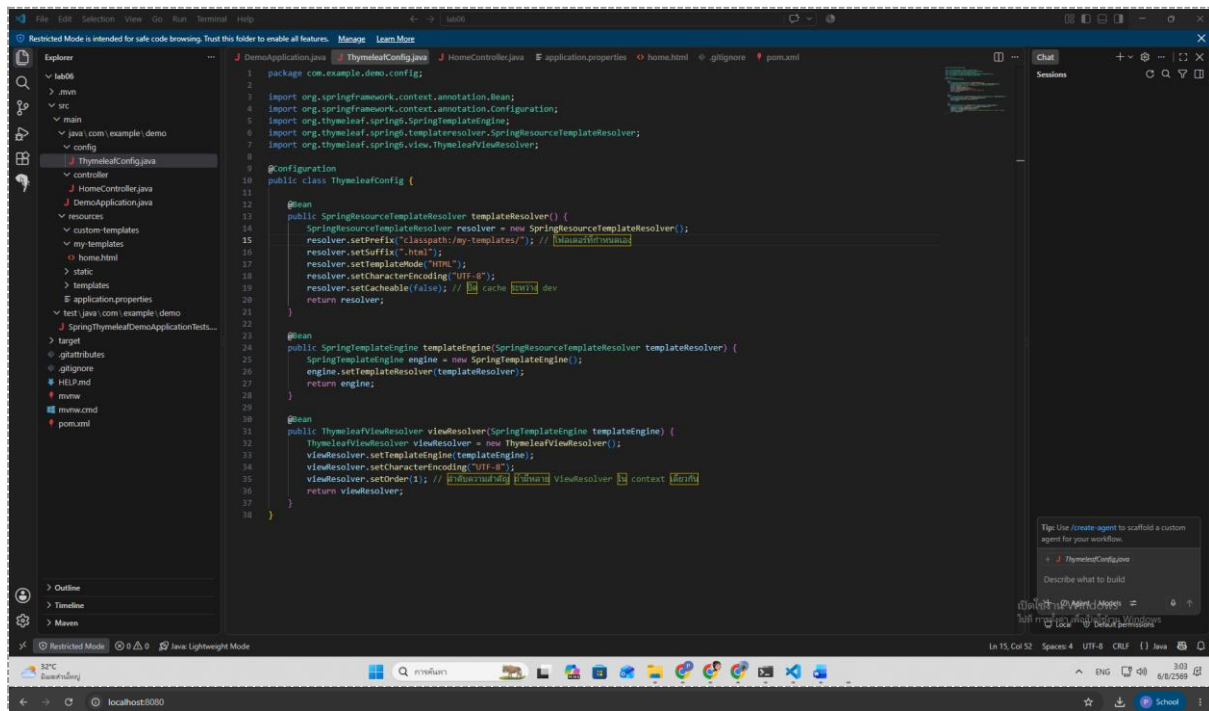
1. แก้ไขค่าตัวแปร `message` ใน `HomeController.java` ให้แสดงชื่อ-นามสกุลของตัวเองแทนข้อความเดิม แล้ว capture หน้าเว็บ `http://localhost:8080/` ที่แสดงชื่อของตัวเอง (3 คะแนน)



2. เพิ่มตัวแปรใหม่ชื่อ `studentId` ส่งจาก Controller ไปแสดงต่อจากชื่อในหน้า `home.html` (เช่น "สวัสดี ชื่อ (รหัส XXX)") แล้ว capture หน้าเว็บที่แสดงผล (3 คะแนน)



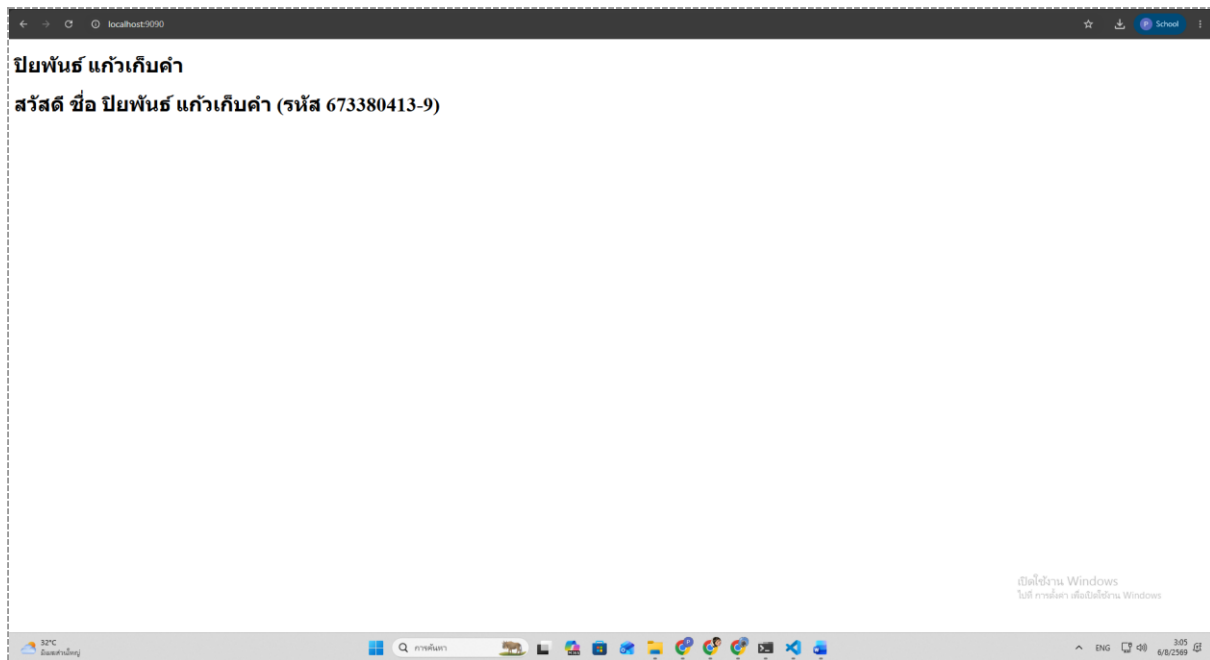
3. เปลี่ยนค่า resolver.setPrefix() ใน ThymeleafConfig.java ให้ชี้ไปโฟลเดอร์ใหม่ชื่อ my-templates/ แทน custom-templates/ (ต้องย้ายไฟล์ home.html ไปด้วย) แล้ว capture ทั้งโค้ดที่แก้ และหน้าเว็บที่ยังทำงานปกติ (4 คะแนน)



ปัยพันธ์ แก้วเกศคำ

สวัสดี ชื่อ ปัยพันธ์ แก้วเกศคำ (รหัส 673380413-9)

4. เปลี่ยน server.port ใน application.properties เป็น 9090 แล้ว capture หน้าเว็บที่ URL เปลี่ยนเป็น http://localhost:9090/ (3 คะแนน)



5. เพิ่ม view ใหม่ชื่อ "about" (เพิ่ม @GetMapping("/about") และไฟล์ about.html)

ให้แสดงข้อความแนะนำตัวสั้นๆ แล้ว capture หน้าเว็บ /about ที่แสดงผลลัพธ์ (4 คะแนน)

